

nginx JS — Dynamic Reconfiguration Guide (servers)

Table of Contents

1. Overview	2
1.1. The two-phase commit model	3
1.2. What this guide covers	3
2. Inspecting servers	3
2.1. <code>nginx.http.servers[]</code>	4
2.2. Server identity properties	4
2.3. Finding a server by name	4
2.4. <code>nginx.http.match(uri [, serverName])</code>	4
3. Adding servers	5
3.1. <code>nginx.http.addServer(name [, opts])</code>	5
3.2. Difference between <code>addServer</code> and <code>clone</code>	5
3.3. Example: add a server and populate locations	6
3.4. Example: add server from template	6
3.5. Flow diagram: <code>addServer</code> two-phase	6
4. Removing servers	7
4.1. <code>nginx.http.removeServer(name)</code>	7
4.2. Example: dynamic server lifecycle (add → serve → remove)	7
4.3. Flow diagram: <code>removeServer</code> two-phase	8
5. Cloning a server	8
5.1. <code>srv.clone(newName)</code>	8
5.2. Shared location structs after clone	8
5.3. Example: blue-green deployment by server clone	9
6. Managing server names	10
6.1. <code>srv.setNames(namesArray)</code>	10
6.2. Example: certificate rollover with name swap	11
7. Reconfiguring server properties	11
7.1. Writable server properties	11
7.2. Read-only server properties	12
7.3. Example: runtime header policy change	12
8. SSL reconfiguration	12
8.1. The <code>NginxSSL</code> sub-object	12
8.2. SSL read-only properties	12
8.3. SSL read/write properties	13
8.4. <code>ssl.setCertificate(certPath, keyPath)</code>	13

8.5. <code>ssl.setProtocols(protocolArray)</code>	13
8.6. <code>ssl.setCiphers(cipherString)</code>	14
8.7. Example: zero-downtime certificate rotation	14
8.8. Example: emergency protocol downgrade	14
9. Complete examples	15
9.1. Example 1: per-tenant server provisioning	15
9.2. Example 2: atomic blue-green server swap	16
9.3. Example 3: certificate rotation endpoint	17
9.4. Example 4: server inventory endpoint	17
9.5. Example 5: validate routing before going live	18
10. Atomicity and cross-worker propagation	19
10.1. Single-worker atomicity	19
10.2. Cross-worker atomicity	19
10.3. Flow diagram: cross-worker <code>addServer</code> + <code>rebuildVhostDispatch</code>	20
11. API Reference	21
11.1. <code>nginx.http.servers[]</code>	21
11.2. <code>nginx.http.addServer(name [, opts])</code>	21
11.3. <code>nginx.http.removeServer(name)</code>	21
11.4. <code>nginx.http.rebuildVhostDispatch()</code>	21
11.5. <code>nginx.http.match(uri [, serverName])</code>	21
11.6. <code>srv.clone(newName)</code>	22
11.7. <code>srv.setNames(namesArray)</code>	22
11.8. <code>ssl.setCertificate(certPath, keyPath)</code>	22
11.9. <code>ssl.setProtocols(protocolArray)</code>	22
11.10. <code>ssl.setCiphers(cipherString)</code>	23
12. Design notes	23
12.1. Why <code>rebuildVhostDispatch()</code> is explicit	23
12.2. <code>addServer</code> vs <code>srv.clone</code> : location tree sharing	23
12.3. Certificate rotation and existing sessions	23
12.4. <code>rebuildVhostDispatch</code> and per-worker hashes	23
12.5. Duplicate server names	23



DRAFT FOR COMMENTS — This document is a work in progress. Content, API names, and examples are subject to change before release.

1. Overview

This guide covers runtime management of nginx **server blocks** via the JavaScript DOM: adding, removing, renaming, and reconfiguring virtual servers, including their SSL configuration.

Prerequisites: read the *for single worker* guide first for `write_mode`, `snapshot()`, and `suspendAcceptance()`; and the *locations* guide for `addLocation()`, `removeLocation()`, and `clone()`.

1.1. The two-phase commit model

Server-level structural operations (add, remove, rename) follow a **two-phase** model:

Phase 1 – Structural change:

```
nginx.http.addServer(name)      ← server added to nginx.http.servers[]
nginx.http.removeServer(name)   ← server removed from nginx.http.servers[]
srv.setNames([...])            ← server_name list updated in memory
```

After Phase 1: `nginx.http.servers[]` reflects the change, but the server-name hash tables are NOT yet updated. Real HTTP requests still route according to the OLD hash.

Phase 2 – Activate routing:

```
nginx.http.rebuildVhostDispatch()
```

After Phase 2: the server-name hash is rebuilt from all current `nginx.http.servers[]`. New requests are dispatched to the updated set.

This is intentional: it lets you prepare a server completely (add locations, set handler, configure SSL) before any request can reach it.



`addLocation()` and `removeLocation()` (location-level changes) do **not** require a `rebuildVhostDispatch()` call — they update the location BST immediately.

1.2. What this guide covers

Topic	API
Inspect servers	<code>nginx.http.servers[], srv.name, srv.names, srv.locations</code>
Add server	<code>nginx.http.addServer(name [, opts]) + rebuildVhostDispatch()</code>
Remove server	<code>nginx.http.removeServer(name) + rebuildVhostDispatch()</code>
Clone server	<code>srv.clone(newName) + rebuildVhostDispatch()</code>
Rename / multi-name	<code>srv.setNames([...]) + rebuildVhostDispatch()</code>
Reconfigure properties	<code>srv.root, srv.clientHeaderTimeout, srv.mergeSlashes, etc.</code>
SSL reconfiguration	<code>srv.ssl.setCertificate(), ssl.setProtocols(), ssl.setCiphers()</code>
Test routing	<code>nginx.http.match(uri [, serverName])</code>
Atomicity	<code>suspendAcceptance(), suspendAllWorkers()</code>

2. Inspecting servers

2.1. `nginx.http.servers[]`

The live array of all `NginxServer` objects registered on the HTTP module. Includes both static (from `nginx.conf`) and dynamically added servers.

```
// List all servers and their names
nginx.http.servers.forEach((srv, i) => {
  nginx.log(`servers[${i}]: ${srv.names.join(', ')} (${srv.locations.length}
locations)`);
});
```

2.2. Server identity properties

Property	Access	Description
<code>name</code>	read-only	First <code>server_name</code> string, or <code>""</code> if none.
<code>names</code>	read-only	Array of all <code>server_name</code> strings for this server.
<code>root</code>	read/write	Document root from the server's implicit <code>/</code> location.
<code>locations</code>	read-only	Array of <code>NginxLocation</code> objects (see <i>locations</i> guide).

2.3. Finding a server by name

```
function findServer(name) {
  return nginx.http.servers.find(srv =>
    srv.names.some(n => n.toLowerCase() === name.toLowerCase())
  ) || null;
}

const api = findServer('api.example.com');
if (api) {
  nginx.log('api root: ' + api.root);
}
```

2.4. `nginx.http.match(uri [, serverName])`

Simulates nginx's location dispatch algorithm on the JavaScript side. Returns the `NginxLocation` that would handle `uri`, or `null` if no location matches.

```
// Test location dispatch on the first server
const loc = nginx.http.match('/api/v2/users');
nginx.log(loc ? 'matched: ' + loc.path : 'no match');

// Test on a specific virtual host
```

```
const loc2 = nginx.http.match('/static/img.png', 'cdn.example.com');
```

Useful for verifying that `addLocation()` / `removeLocation()` changes route requests as expected before enabling them for real traffic.

```
match('/api/v2/users', 'api.example.com'):
  Scans prefix_locs[] → /api/v2/ (longest prefix match)
  Checks regex_locs[] → no match
  Returns: NginxLocation { path: '/api/v2/', matchType: 'prefix' }
```

3. Adding servers

3.1. `nginx.http.addServer(name [, opts])`

Creates a new virtual server with `name` as its primary `server_name`. The server is added to `nginx.http.servers[]` and to every listening address's vhost dispatch table, but routing does not activate until `rebuildVhostDispatch()` is called.

```
const newSrv = nginx.http.addServer('new.example.com');
// ... configure locations, handler, etc. ...
nginx.http.rebuildVhostDispatch(); // activate routing
```

Options:

Option	Description
<code>template: 'name'</code>	Use the named server as the configuration template (inherits server-level defaults such as <code>clientHeaderTimeout</code> , <code>mergeSlashes</code> , etc.). Defaults to the first server in <code>nginx.http.servers[]</code> . The new server starts with an empty location tree regardless of the template's locations — use <code>srv.clone()</code> if you need to copy locations.

3.2. Difference between `addServer` and `clone`

	<code>nginx.http.addServer(name, {template})</code>	<code>srv.clone(newName)</code>
Location tree	Empty (no locations)	Full copy of source locations
Config values	Inherited from template	Copied from source (shallow copy)
Location clcf sharing	— (no locations yet)	Shared initially; independent after <code>addLocation()</code>
Use case	Build a new server from scratch	Duplicate an existing server

3.3. Example: add a server and populate locations

```
// js_source: provision.js

const srv = nginx.http.addServer('tenant-acme.example.com');

// Document root
srv.root = '/var/www/tenants/acme';

// Health endpoint
const health = srv.addLocation('= /health');
health.handler = (req) => {
    req.respond(200, {}, 'OK');
};

// Static files
const static_ = srv.addLocation('/static/');
static_.root = '/var/www/tenants/acme/static';
static_.sendfile = true;

// Activate routing
nginx.http.rebuildVhostDispatch();
```

3.4. Example: add server from template

```
// Inherit clientHeaderTimeout, mergeSlashes, etc. from 'base.example.com'
const newSrv = nginx.http.addServer('beta.example.com',
    { template: 'base.example.com' });

// Add locations specific to the beta server
const beta = newSrv.addLocation('/beta/');
beta.handler = betaHandler;

nginx.http.rebuildVhostDispatch();
```

3.5. Flow diagram: addServer two-phase

Phase 1: `nginx.http.addServer('new.example.com')`

`nginx.http.servers[]` `vhost dispatch tables`

`[0] example.com` `addr_conf->virtual_names`

```
[1] api.example.com      hash: example.com → [0]
[2] new.example.com ←NEW  api.example.com → [1]
                           ← hash NOT YET updated
                           ← requests for new.example.com →
                           no match → fall back to default
```

Phase 2: `nginx.http.rebuildVhostDispatch()`

```
vhost dispatch tables rebuilt:
  hash: example.com      → [0]
        api.example.com → [1]
        new.example.com → [2] ← NOW ACTIVE
```

4. Removing servers

4.1. `nginx.http.removeServer(name)`

Removes the server whose primary name (case-insensitive) matches `name`. Splices it from `nginx.http.servers[]` and from every listening address's vhost entry list. Returns `true` if found and removed, `false` otherwise.

Must call `rebuildVhostDispatch()` to stop routing requests to the removed server.

```
const removed = nginx.http.removeServer('old.example.com');
if (removed) {
  nginx.http.rebuildVhostDispatch();
}
```



In-flight requests being processed by the removed server's handlers complete normally. The server's config memory is not freed until the server opaque object is garbage-collected.

4.2. Example: dynamic server lifecycle (add → serve → remove)

```
// Bring up a temporary server for a maintenance window
async function withTemporaryServer(name, handler, durationMs) {
  const srv = nginx.http.addServer(name);
  const loc = srv.addLocation('/');
  loc.handler = handler;
  nginx.http.rebuildVhostDispatch();

  nginx.log('temporary server live: ' + name);
  await nginx.setTimeout(durationMs);
}
```

```

    nginx.http.removeServer(name);
    nginx.http.rebuildVhostDispatch();
    nginx.log('temporary server removed: ' + name);
}

// Usage: maintenance page for 2 minutes
withTemporaryServer('maintenance.example.com',
    (req) => { req.respond(503, {}, 'Maintenance in progress'); },
    120000);

```

4.3. Flow diagram: removeServer two-phase

```

Before removeServer('api.example.com'):
  servers[]: example.com, api.example.com, cdn.example.com
  hash: all three names routable

Phase 1: nginx.http.removeServer('api.example.com')
  servers[]: example.com, cdn.example.com
  hash: still routes api.example.com → (now stale pointer)
  ← DANGER: do not leave this state for long

Phase 2: nginx.http.rebuildVhostDispatch()
  hash rebuilt: example.com, cdn.example.com
  Requests for api.example.com → no match → default server

```

5. Cloning a server

5.1. srv.clone(newName)

Creates an independent copy of the server under `newName`. Unlike `addServer()`, the clone inherits the **full current location tree** — all prefix, regex, and named locations (both static and dynamic).

```

const primary = findServer('example.com');
const mirror  = primary.clone('mirror.example.com');
nginx.http.rebuildVhostDispatch();

```

5.2. Shared location structs after clone

After `clone()`, the source and clone initially share the underlying `ngx_http_core_loc_conf_t` C structs for each copied location. This means:

- Reading `loc.root` / `loc.handler` / etc. on either server returns the same value.
- Writing `loc.handler = fn` via a location object from either server modifies the **shared** struct and

affects **both** servers.

- Calling `clone.addLocation('/new/')` creates a new, independent entry only in the clone's BST. It does not affect the source.

To make a location's config truly independent after cloning, use `loc.snapshot()` on the clone's location and then immediately `restore()` — this forces a copy-on-write. Or set the field you want to differ: the setter will write to the shared struct (which is still correct if you intend to update both), or use `write_mode='local'` for per-request isolation.

```
srv.clone('mirror.example.com');
```

primary.locations:		mirror.locations (after clone):
/api/ → clcf_A	—————	/api/ → clcf_A (shared pointer)
/static/ → clcf_B	—————	/static/ → clcf_B (shared pointer)
/ → clcf_C	—————	/ → clcf_C (shared pointer)

```
mirror.addLocation('/mirror-only/'):
  /mirror-only/ → clcf_NEW ← independent, only in mirror
```

```
primary.findLocation('/api/').handler = v2; ← modifies clcf_A
→ affects BOTH primary and mirror /api/
```

5.3. Example: blue-green deployment by server clone

```
// js_source: blue_green.js

let activeName = 'app.example.com';
let standbyName = 'app-standby.internal';

// Initial state: active server is live
const active = findServer(activeName);

async function deployGreen(newHandlers) {
  // Clone active → standby (inherits all locations and settings)
  const standby = active.clone(standbyName);

  // Update handlers on standby
  // (sets shared clcf, visible on both servers – that is fine here
  // because standby is not yet routable)
  newHandlers.forEach(({ path, handler }) => {
    const loc = standby.findLocation(path);
    if (loc) { loc.handler = handler; }
  });

  // Add standby to routing
  nginx.http.rebuildVhostDispatch();
  nginx.log('standby live at: ' + standbyName);
}
```

```
// Smoke-test the standby via match() before cutover
const health = nginx.http.match('/health', standbyName);
nginx.log('standby health loc: ' + (health ? health.path : 'none'));

// Cutover: swap names
await nginx.suspendAllWorkers();
try {
    active.setNames([standbyName]);    // demote active to standby
    standby.setNames([activeName]);    // promote standby to active
    nginx.http.rebuildVhostDispatch();
} finally {
    await nginx.resumeAllWorkers();
}

nginx.log('cutover complete: ' + activeName + ' → new handlers');
}
```

6. Managing server names

6.1. `srv.setNames(namesArray)`

Replaces the `server_name` list for this server. Strings are copied into the cycle pool. The change is visible immediately via `srv.names` and `srv.name`, but routing is not updated until `rebuildVhostDispatch()` is called.

```
const srv = findServer('example.com');

// Add an alias
srv.setNames(['example.com', 'www.example.com', 'example.net']);
nginx.http.rebuildVhostDispatch();
```

```
// Remove all names (server becomes the default_server for its address)
srv.setNames([]);
nginx.http.rebuildVhostDispatch();
```



`setNames` replaces the entire name list. To add a single alias, read `srv.names`, push to it, and pass the updated array back.

```
function addAlias(srv, alias) {
    const names = srv.names.slice();    // copy current names
    if (!names.includes(alias)) {
        names.push(alias);
        srv.setNames(names);
        nginx.http.rebuildVhostDispatch();
    }
}
```

```

}

function removeAlias(srv, alias) {
  const names = srv.names.filter(n => n !== alias);
  srv.setNames(names);
  nginx.http.rebuildVhostDispatch();
}

```

6.2. Example: certificate rollover with name swap

Wildcard certificates covering `*.example.com` can be rotated by swapping which server responds to each name:

```

async function swapServerNames(nameA, nameB) {
  const srvA = findServer(nameA);
  const srvB = findServer(nameB);

  if (!srvA || !srvB) { throw new Error('server not found'); }

  await nginx.suspendAllWorkers();
  try {
    const tempA = srvA.names.slice();
    const tempB = srvB.names.slice();

    srvA.setNames(tempB);
    srvB.setNames(tempA);
    nginx.http.rebuildVhostDispatch();
  } finally {
    await nginx.resumeAllWorkers();
  }
}

```

7. Reconfiguring server properties

7.1. Writable server properties

Property	Type	Description
<code>root</code>	string	Document root for the implicit <code>/</code> location.
<code>clientHeaderBufferSize</code>	number (bytes)	Size of buffer for reading client request headers.
<code>clientHeaderTimeout</code>	number (ms)	Timeout for reading the full HTTP request header.
<code>ignoreInvalidHeaders</code>	boolean	Whether to ignore invalid header names (default: <code>true</code>).

Property	Type	Description
<code>mergeSlashes</code>	boolean	Whether to merge consecutive slashes in URIs (default: <code>true</code>).
<code>underscoresInHeaders</code>	boolean	Whether to allow underscores in header names (default: <code>false</code>).
<code>serverTokens</code>	'on' 'off' 'build'	Controls the <code>Server:</code> response header.
<code>connectionPoolSize</code>	number (bytes)	Size of the per-connection memory pool.
<code>requestPoolSize</code>	number (bytes)	Size of the per-request memory pool.

7.2. Read-only server properties

Property	Description
<code>largeClientHeaderBuffers</code>	Object with <code>count</code> and <code>size</code> (bytes). Set via <code>nginx.conf</code> only.
<code>ssl</code>	<code>NginxSSL</code> sub-object, or <code>null</code> if SSL is not configured. SSL settings are mutated via <code>ssl.setCertificate()</code> etc.

7.3. Example: runtime header policy change

```
const srv = findServer('legacy.example.com');

// Legacy API clients send underscores in header names
nginx.suspendAcceptance();
srv.underscoresInHeaders = true;
srv.ignoreInvalidHeaders = false;
nginx.resumeAcceptance();
```

8. SSL reconfiguration

8.1. The NginxSSL sub-object

`srv.ssl` returns a `NginxSSL` wrapper if the server has `ssl_certificate` configured, or `null` otherwise.

```
const ssl = findServer('secure.example.com').ssl;
if (!ssl) { throw new Error('no SSL configured'); }
```

8.2. SSL read-only properties

Property	Description
<code>certificate</code>	Current certificate file path (string).
<code>certificateKey</code>	Current private key file path (string).
<code>protocols</code>	Array of enabled protocol strings (e.g. <code>['TLSv1.2', 'TLSv1.3']</code>).
<code>ciphers</code>	OpenSSL cipher string.
<code>verify</code>	Client certificate verification mode (<code>'off'</code> , <code>'on'</code> , <code>'optional'</code> , <code>'optional_no_ca'</code>).
<code>clientCertificate</code>	Path to client CA bundle (or <code>""</code> if not set).
<code>trustedCertificate</code>	Path to trusted CA bundle (or <code>""</code> if not set).
<code>ecdhCurve</code>	ECDH curve name or <code>"auto"</code> .
<code>dhparam</code>	DH parameters file path (or <code>""</code> if not set).

8.3. SSL read/write properties

Property	Type	Description
<code>sessionTimeout</code>	number (seconds)	TLS session cache timeout.
<code>sessionTickets</code>	boolean	Whether to use TLS session tickets.
<code>preferServerCiphers</code>	boolean	Whether to prefer the server's cipher order.
<code>verifyDepth</code>	number	Maximum depth of client certificate chain.

8.4. `ssl.setCertificate(certPath, keyPath)`

Hot-swaps the TLS certificate and private key on the live `SSL_CTX`. Calls `SSL_CTX_use_certificate_chain_file()` and `SSL_CTX_use_PrivateKey_file()` (PEM format), then validates that the cert and key match.

Takes effect for all **new** TLS handshakes immediately. Existing sessions continue with the old certificate until they expire. Throws on any error (file not found, invalid format, key mismatch).

```
const ssl = findServer('secure.example.com').ssl;

ssl.setCertificate(
  '/etc/ssl/certs/2025/secure.example.com.pem',
  '/etc/ssl/private/2025/secure.example.com.key'
);
nginx.log('certificate rotated: ' + ssl.certificate);
```

8.5. `ssl.setProtocols(protocolArray)`

Sets the enabled TLS protocol versions on the live `SSL_CTX`.

Accepted values: 'SSLv2', 'SSLv3', 'TLSv1', 'TLSv1.1', 'TLSv1.2', 'TLSv1.3'.

```
// Enforce TLS 1.2+ only
ssl.setProtocols(['TLSv1.2', 'TLSv1.3']);

// Re-enable TLS 1.1 for legacy clients temporarily
ssl.setProtocols(['TLSv1.1', 'TLSv1.2', 'TLSv1.3']);
```

8.6. ssl.setCiphers(cipherString)

Sets the OpenSSL cipher suite string on the live `SSL_CTX`.

```
ssl.setCiphers('ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256');
```

8.7. Example: zero-downtime certificate rotation

```
loc.handler = async (req) => {
  const body    = JSON.parse(await req.readBody());
  const srvName = body.server;
  const certPath = body.cert;
  const keyPath  = body.key;

  const srv = findServer(srvName);
  if (!srv) { req.respond(404, {}, 'server not found'); return; }

  const ssl = srv.ssl;
  if (!ssl) { req.respond(400, {}, 'server has no SSL'); return; }

  const oldCert = ssl.certificate;
  const oldKey  = ssl.certificateKey;

  try {
    ssl.setCertificate(certPath, keyPath);
    nginx.log(`cert rotated on ${srvName}: ${oldCert} → ${certPath}`);
    req.json({ rotated: true, cert: ssl.certificate });
  } catch (e) {
    req.respond(500, {}, 'cert rotation failed: ' + e.message);
  }
};
```

8.8. Example: emergency protocol downgrade

Allow older TLS versions temporarily for a critical client integration:

```
loc.handler = async (req) => {
```

```

const ssl      = findServer('api.example.com').ssl;
const original = ssl.protocols.slice();    // save current protocols

// Allow TLS 1.1 for 10 minutes, then revert
ssl.setProtocols(['TLSv1.1', 'TLSv1.2', 'TLSv1.3']);
nginx.log('TLS 1.1 re-enabled temporarily');

await nginx.setTimeout(600000);    // 10 min

ssl.setProtocols(original);
nginx.log('TLS protocols restored: ' + original.join(', '));
};

```



`ssl.protocols` returns the current enabled protocol list. Saving and restoring it is equivalent to using `snapshot()` on a sub-object — an explicit save/restore is required here because `NginxSSL` does not currently support `snapshot()`.

9. Complete examples

9.1. Example 1: per-tenant server provisioning

```

// js_source: tenants.js

const templateServer = nginx.http.servers[0];

function provisionTenant(tenant) {
  const name = tenant.id + '.example.com';

  // Idempotent: if already exists, return existing
  const existing = findServer(name);
  if (existing) { return existing; }

  const srv = nginx.http.addServer(name);
  srv.root = '/var/www/tenants/' + tenant.id;

  // Standard locations
  const health = srv.addLocation('= /health');
  health.handler = (req) => { req.respond(200, {}, 'OK'); };

  const api = srv.addLocation('/api/');
  api.handler = makeTenantApiHandler(tenant);

  const static_ = srv.addLocation('/static/');
  static_.root = '/var/www/static';
  static_.sendfile = true;

  nginx.http.rebuildVhostDispatch();
}

```

```

    nginx.log('provisioned: ' + name);
    return srv;
}

function deprovisionTenant(tenantId) {
    const name = tenantId + '.example.com';
    const removed = nginx.http.removeServer(name);
    if (removed) {
        nginx.http.rebuildVhostDispatch();
        nginx.log('deprovisioned: ' + name);
    }
    return removed;
}

```

9.2. Example 2: atomic blue-green server swap

```

let blueActive = true;

loc.handler = async (req) => {
    const body = JSON.parse(await req.readBody());
    const flavor = body.flavor; // 'blue' or 'green'
    const target = 'app.example.com';

    // Build the new (inactive) server with the new handler
    const newSrv = nginx.http.addServer(target + '-' + flavor);
    const apiLoc = newSrv.addLocation('/api/');
    apiLoc.handler = flavor === 'blue' ? blueHandler : greenHandler;

    // Verify new server routes correctly via match() before cutover
    const matched = nginx.http.match('/api/test', target + '-' + flavor);
    if (!matched || !matched.handler) {
        nginx.http.removeServer(target + '-' + flavor);
        nginx.http.rebuildVhostDispatch();
        req.respond(500, {}, 'route test failed');
        return;
    }

    // Atomic cutover: suspend all workers, swap names, rebuild
    await nginx.suspendAllWorkers();
    try {
        const oldSrv = findServer(target);
        if (oldSrv) {
            oldSrv.setNames([target + '-old']);
        }
        newSrv.setNames([target]);
        nginx.http.rebuildVhostDispatch();
        blueActive = (flavor === 'blue');
    } finally {
        await nginx.resumeAllWorkers();
    }
}

```



```

}

// Clean up the old server after traffic drains
await nginx.setTimeout(30000);
nginx.http.removeServer(target + '-old');
nginx.http.rebuildVhostDispatch();

req.json({ active: flavor });
};

```

9.3. Example 3: certificate rotation endpoint

```

// POST /admin/rotate-cert
// Body: { "server": "secure.example.com", "cert": "/path/cert.pem", "key":
"/path/key.pem" }

loc.handler = async (req) => {
  const body = JSON.parse(await req.readBody());
  const srv = findServer(body.server);

  if (!srv || !srv.ssl) {
    req.respond(404, {}, 'server or SSL not found');
    return;
  }

  const ssl = srv.ssl;
  const oldCert = ssl.certificate;

  try {
    ssl.setCertificate(body.cert, body.key);
    req.json({
      server: body.server,
      old: oldCert,
      new: ssl.certificate,
      rotated: true,
    });
  } catch (e) {
    req.respond(500, {}, e.message);
  }
};

```

9.4. Example 4: server inventory endpoint

```

const inv = nginx.http.servers[0].addLocation('= /__servers');
nginx.http.rebuildVhostDispatch();

inv.handler = (req) => {

```

```

const data = nginx.http.servers.map(srv => ({
  name:      srv.name,
  names:     srv.names,
  root:      srv.root,
  locationCount: srv.locations.length,
  hasSSL:    srv.ssl !== null,
  sslCert:   srv.ssl ? srv.ssl.certificate : null,
  sslProtocols: srv.ssl ? srv.ssl.protocols : null,
  serverTokens: srv.serverTokens,
}));
req.json({ servers: data, count: data.length });
};

```

9.5. Example 5: validate routing before going live

Use `nginx.http.match()` to assert location dispatch is correct before calling `rebuildVhostDispatch()`:

```

function validateRouting(srv) {
  const cases = [
    { uri: '/health',      expect: '= /health' },
    { uri: '/api/users',   expect: '/api/' },
    { uri: '/static/a.js', expect: '/static/' },
    { uri: '/unknown/path', expect: '/' },
  ];

  const errors = [];

  cases.forEach(({ uri, expect }) => {
    const loc = nginx.http.match(uri, srv.name);
    if (!loc || loc.path !== expect) {
      errors.push(`${uri}: expected '${expect}' got '${loc ? loc.path :
'null'}'`);
    }
  });

  return errors;
}

// Usage before activating a new server:
const newSrv = nginx.http.addServer('new.example.com');
// ... add locations ...

const errors = validateRouting(newSrv);
if (errors.length > 0) {
  nginx.http.removeServer('new.example.com');
  throw new Error('Routing validation failed:\n' + errors.join('\n'));
}

```

```
nginx.http.rebuildVhostDispatch();
```

10. Atomicity and cross-worker propagation

10.1. Single-worker atomicity

Property writes on `NginxServer` and `NginxSSL` objects are individual assignments. Use `suspendAcceptance()` to make a multi-field update appear atomic to new requests on this worker:

```
nginx.suspendAcceptance();

srv.clientHeaderTimeout    = 10000;
srv.underscoresInHeaders   = true;
srv.ignoreInvalidHeaders   = false;

nginx.resumeAcceptance();
```

10.2. Cross-worker atomicity

Server structural changes and `rebuildVhostDispatch()` affect only the current worker's server-name hash. To propagate to all workers:

```
const cfgWorker = new SharedWorker('cfgworker.js');

// Each worker applies addServer fan-out from the SharedWorker:
cfgWorker.onmessage = (msg) => {
  if (msg.data.op === 'addServer') {
    const srv = nginx.http.addServer(msg.data.name, msg.data.opts);
    // Re-apply locations/handlers from registry (functions can't be cloned)
    msg.data.locationDefs.forEach(def => {
      const loc = srv.addLocation(def.pattern);
      if (def.root) { loc.root = def.root; }
      if (def.handler) { loc.handler = handlerRegistry[def.handler]; }
    });
    nginx.http.rebuildVhostDispatch();
  } else if (msg.data.op === 'removeServer') {
    nginx.http.removeServer(msg.data.name);
    nginx.http.rebuildVhostDispatch();
  }
};
```

```
// Handler: add server across all workers atomically
loc.handler = async (req) => {
  const cfg = JSON.parse(await req.readBody());
```

```

await nginx.suspendAllWorkers();
try {
  // Apply locally
  const srv = nginx.http.addServer(cfg.name, cfg.opts);
  // ... add locations ...
  nginx.http.rebuildVhostDispatch();

  // Broadcast to all other workers
  cfgWorker.postMessage({
    op: 'addServer',
    name: cfg.name,
    opts: cfg.opts,
    locationDefs: cfg.locationDefs,
  });

  req.json({ added: cfg.name });
} finally {
  await nginx.resumeAllWorkers();
}
};

```

10.3. Flow diagram: cross-worker addServer + rebuildVhostDispatch



11. API Reference

11.1. `nginx.http.servers[]`

Read-only live array of all `NginxServer` objects.

```
NginxServer[]  nginx.http.servers
```

11.2. `nginx.http.addServer(name [, opts])`

Creates a new virtual server with an empty location tree.

```
NginxServer  nginx.http.addServer(name)
NginxServer  nginx.http.addServer(name, { template: 'existingName' })
```

Does NOT activate routing — call `rebuildVhostDispatch()` after.

Throws `TypeError` if `name` is missing. Throws `InternalError` if no existing servers or no `http{}` block.

11.3. `nginx.http.removeServer(name)`

Removes the server matching `name` (case-insensitive).

```
boolean  nginx.http.removeServer(name)
```

Returns `true` if removed, `false` if not found. Does NOT update routing — call `rebuildVhostDispatch()` after.

11.4. `nginx.http.rebuildVhostDispatch()`

Rebuilds the server-name hash tables for all listening addresses. Must be called after `addServer()`, `removeServer()`, or `srv.setNames()`.

```
undefined  nginx.http.rebuildVhostDispatch()
```

Throws `InternalError` on hash allocation or init failure.

11.5. `nginx.http.match(uri [, serverName])`

Simulates nginx's location dispatch algorithm. Returns the `NginxLocation` that would handle `uri`, or `null` on no match.

```
NgixLocation | null  nginx.http.match(uri)
NgixLocation | null  nginx.http.match(uri, serverName)
```

Uses the first server in `nginx.http.servers[]` when `serverName` is omitted. Performs the full exact → prefix → ^~ → regex → fallback sequence on the JavaScript-side location arrays.

11.6. `srv.clone(newName)`

Creates an independent copy of the server (with full location tree).

```
NgixServer  srv.clone(newName)
```

Does NOT activate routing — call `rebuildVhostDispatch()` after.

11.7. `srv.setNames(namesArray)`

Replaces the `server_name` list.

```
undefined  srv.setNames([ 'name1', 'name2', ... ])
```

Does NOT update routing — call `rebuildVhostDispatch()` after.

Throws `TypeError` if the argument is not an array.

11.8. `ssl.setCertificate(certPath, keyPath)`

Hot-swaps the TLS certificate and private key on the live `SSL_CTX`.

```
undefined  ssl.setCertificate(certPath, keyPath)
```

Takes effect for new handshakes immediately. Throws on file I/O error, PEM parse error, or cert/key mismatch.

11.9. `ssl.setProtocols(protocolArray)`

Sets enabled TLS protocol versions on the live `SSL_CTX`.

```
undefined  ssl.setProtocols(['TLSv1.2', 'TLSv1.3'])
```

Accepted strings: `'SSLv2'`, `'SSLv3'`, `'TLSv1'`, `'TLSv1.1'`, `'TLSv1.2'`, `'TLSv1.3'`.

11.10. `ssl.setCiphers(cipherString)`

Sets the OpenSSL cipher suite string.

```
undefined  ssl.setCiphers(opensslCipherString)
```

12. Design notes

12.1. Why `rebuildVhostDispatch()` is explicit

nginx's server-name dispatch is based on hash tables built at startup. A single atomic rebuild — rather than incremental insertion — avoids hash collisions, wildcard ordering issues, and PCRE regex recompilation during partial updates. The explicit call gives the JS script control over exactly when the new routing becomes active.

12.2. `addServer` vs `srv.clone`: location tree sharing

`nginx.http.addServer()` always creates an empty location tree. This is deliberate: it forces the caller to explicitly define all locations for the new server, preventing unexpected inheritance of handler functions that may reference per-server state.

`srv.clone()` copies the full tree for cases where the intent is to create a near-identical virtual host. The initial struct sharing is safe because the clone is not yet reachable until `rebuildVhostDispatch()` is called, giving the caller a window to customise the clone before any request can reach it.

12.3. Certificate rotation and existing sessions

`ssl.setCertificate()` replaces the certificate on the live `SSL_CTX` using `SSL_CTX_use_certificate_chain_file` / `SSL_CTX_use_PrivateKey_file`. This is the standard OpenSSL hot-swap path. Sessions established before the swap continue with the old certificate via OpenSSL's session cache or tickets; only new handshakes use the new certificate.

12.4. `rebuildVhostDispatch` and per-worker hashes

Each worker maintains its own server-name hash. A `rebuildVhostDispatch()` call on Worker 0 does not update Workers 1 ... N. Broadcast via `SharedWorker` (see cross-worker section above) is required to synchronise all workers.

12.5. Duplicate server names

`rebuildVhostDispatch()` logs a warning (`NGX_LOG_WARN`) and skips duplicate server names silently — it does not throw. The first server registered with a given name wins. Ensure names are unique before calling `addServer()` if strict routing is required.